PIBBSS Final Report: Logically Updateless Decision-Making

Martín Soto
Mathematics and Computer Science Department
University of Barcelona
Barcelona, Catalonia, Spain
martinsotoq2911@gmail.com

This progress report includes WIP loose ends (in some areas we are still even questioning which mathematical results to prove), and is stylistically more informal and detailed than the paper we will eventually produce. Abram and I are still working on new technical directions and expansions (and will continue to do so after the fellowship ends), some of which are hinted at in this writing, and others of which are completely ignored here.

1 Introduction

This work is framed inside Agent Foundations, and the general pursuit of understanding idealized rationality. It can be understood as charting the boundaries of how well an embedded agent can perform in online learning, and which kind of mental moves it can make to robustly increase reward.

1.1 Updatelessness

MIRI researchers have been pushing in this direction since the 2010s. As a crystallization of some academically unorthodox ideas about Decision Theory, Wei Dai proposed Updateless Decision Theory (UDT), which is better motivated through the following decision problem:

> **Empirical counterfactual mugging**
>
> Say the entity $\mathbf{\Omega}$, a perfect predictor, throws a fair coin. If it lands Heads, you (the agent $\mathbf{A}$) get to decide whether to pay 100$. If it lands Tails, then $\mathbf{\Omega}$ will perfectly simulate what you would have done in the Heads empirical counterfactual, and will give you 1000$ iff you paid the 100$ in that counterfactual reality.

Mainstream academic decision theories (CDT and EDT), when told they will face such a mugging in the future, choose to precommit to paying the 100$ (if given the chance). Indeed, clearly this has higher expected reward, from this earlier position still uncertain about the outcome of the fair coin, than not paying the 100$. But if they find themselves in the Heads world (the uncertainty is resolved) without having had the opportunity to precommit, they choose not to pay. They have *updated* on the information that the coin landed Heads, and as a consequence they no longer care about the counterfactual Tails world.

This can be understood as a failure of **dynamic stability**. Implicitly, the agent has changed the way it computes expected utility (no longer including the Tails world). And as a consequence, the agent in the Heads world, and the agent in the counterfactual Tails world (or the temporally previous agent before starting the experiment), have different priorities, and cannot cooperate to attain a globally optimal

Submitted to:
PIBBSS Symposium 2023

© Martín Soto
This work is licensed under the
Creative Commons Attribution License.

policy. Different temporal slices of the agent are fighting each other. The motivating drive behind Updatelessness is ensuring dynamic stability. And the basic proposal for doing that is letting a single agent, at the start of the decision sequence, decide on the best global policy (a function from possible future observations to actions).

When we assume logical omniscience (as is usually done in classical Decision Theory), and thus we act **only under empirical uncertainty, the above simple solution just works**. To see that, we still need to solve the question of how to weigh in importance different possible empirical worlds against each other. In Counterfactual mugging, how do we weigh the wealth obtained in the Heads world, versus in the Tails world? Should we care about one more than the other?
As it turns out, implementing a simplicity prior (a mathematical formalization of Occam's razor) is a natural solution to this problem.[1] In such a prior, we "care more" about worlds that are easier to describe, because, intuitively, "they will happen more often" (throughout the multiverse, or any preferred ontology). Here by "easier to describe" we just mean having lower Kolmogorov complexity, that is, being generated by a Turing Machine of shorter description. Such a simplicity prior has the expected behavior for probabilities: both the Heads and the Tails events are as easy to describe, since there are just as many possible empirical worlds satisfying each of the two (that's what it means for them to be equally likely). If the coin were unfair, the situation would change accordingly.

In reality, agents don't only face empirical uncertainty, but also logical uncertainty, that is, uncertainty about the truth of mathematical facts, or even the outputs of finite-time computations. Indeed, consider the following variation on Counterfactual mugging:

> **Logical counterfactual mugging**
> Say the entity $\mathbf{\Omega}$, a perfect predictor, considers (without computing it yet) the millionth digit of π. If it turns out even, you (the agent $\mathbf{A}$) get to decide whether to pay 100$. If it turns out odd, then $\mathbf{\Omega}$ will perfectly simulate what you would have done in the even logical counterfactual, and will give you 1000$ iff you paid the 100$ in that counterfactual reality.

Unfortunately, **the above simple solution doesn't immediately translate to the logically uncertain case**. One simple way to frame this failure is that we don't have any analog of the simplicity prior. That is, we don't have a satisfactory way to assign probabilities to complex logical sentences. There is a clear intuitive sense in which, for the parity of the millionth digit of π, we should go with 50%, because of the strongly believed (but unproved) mathematical conjecture that the digits of π are uniformly pseudo-random. But we don't have a principled way to come up with such guesses for arbitrary formulas. For example, should we also be 50% on the parity of the first digit of π? (That is, be Updateless about its value, despite the ease of computation) And what about the tenth?[2]

[1] In actuality, this solution still leaves us with an arbitrary choice, since there are infinitely many simplicity priors (one for every Universal Turing Machine used to assess Kolmogorov complexity). This of course amounts to specifying our prior over worlds, that is, which realities we expect to inhabit a priori. The practical point can be made that we should choose the right prior to avoid anthropic capture [10], although it can also be argued that we might not want to dogmatically exclude the possibility of inhabiting certain simulations.

[2] The problem of coming up with such guesses can be restated as a Reference Class problem. It "feels natural" to consider that the sentence "the millionth digit of π is even" should be assessed as a member of the Reference Class "the nth digit of π is even". And thus, since we believe the latter Reference Class to have a fraction of true sentences of 0.5 in the limit, we apply that as our guess. But why that Reference Class, and not "the millionth digit of π^n is even"? Why not the Reference Class of all sentences containing the number million? Again, we don't have a principled way to choose.

Another way to frame this failure is that we don't understand logical uncertainty (or logical counterfactuals, or logical possible worlds) as well as we understand empirical uncertainty. In fact, this problem cuts deeper than it might seem: we don't even know how to formalize the above decision problem. Indeed, what do we mean when we say "$\mathbf{\Omega}$ perfectly simulates what you would have done if the millionth digit of π had been even"? If $\mathbf{\Omega}$ just assumes a false sentence and then tries to prove anything about our behavior, it will get logical explosion!
In the empirical case, $\mathbf{\Omega}$ could just take the algorithm that the agent is running to take decisions (for example, our messy brain synapses in the case of humans), feed it different empirical observations, and see how it reacts. There is no logical explosion, since the empirical counterfactual is perfectly consistent (just not what happened in reality). Imagine we do the same for the logical case, so $\mathbf{\Omega}$ simulates the world in which it has told the agent that the digit is actually even. But what if the agent computes the parity in her head (that is, in her algorithm), and so notices that $\mathbf{\Omega}$ is not telling the truth? (And so, maybe, concludes that it's inside a simulation run by $\mathbf{\Omega}$.) In the empirical case there was no a priori way to decide the coin, but now that has become something the agent can internally do. In the "possible world" where indeed the parity is even, that calculation run inside the agent's head would also turn out even. So $\mathbf{\Omega}$ would like to include this in its simulation, by spoofing some of the agent's computations to align with the stated counterfactual. But again, the problem is that we don't have a principled way to do this (to single out "which part of a computation" corresponds to computing that parity). And in fact, it can be argued that such a realist notion of logical uncertainty is impossible in principle, and we'll need other approaches (see Appendix D).

Granted this lack of a realist picture for logical uncertainty, we turn to the best account available of logically uncertain learning: Garrabrant's Logical Inductors [2], presented in Sect. 3.1. They provide a canonical account of both the logically uncertain reasoning of the agent, and the logically counterfactual simulations by $\mathbf{\Omega}$. Using them, we construct a natural framework for logically uncertain decision problems in Sect. 3.2. In it, thinking for longer is represented by running a Logical Inductor for more steps, and some static properties of (allegedly correct) logically uncertain decision-making are re-framed as dynamic learning properties (these are the desiderata of Sect. 2). In Sect. 4 we build an Updateless agent that meets most of our desiderata, and in Sect. 5 we discuss limitations of this construction, and extensions that partly surmount them, as well as the fundamental trade-off between learning and Updatelessness.

While we have focused above in dynamic stability, UDT also aims to accommodate the evidential intuitions that motivate EDT. In particular, we want to entertain the non-cartesian channels of influence between the agent's algorithm and the environment, not mediated through the cartesian route of "the discrete actions chosen by the algorithm". For example, this is the case when $\mathbf{\Omega}$ looks into the agent's code, and decides outcomes for the environment depending on what it's seen. So our work can be understood as an idealized theory of a very general kind of online reinforcement learning, in which such non-cartesian effects of the neural network in the reward function are allowed.

1.2 Theory of change

The obvious upside of this line of research is a better understanding of the decision-theoretic landscape. Indeed, we are starting to explore rigorously some decision-theoretic intuitions that have permeated the Alignment space for quite some time. In trying to construct an $\mathbf{A}$ satisfying the hand-wavy intuitions behind UDT, our main task is charting the boundaries of what is and isn't a priori possible for rational

agents.

The positive ethical consequences of this kind of theoretical better understanding are always of a diffused and unpredictable kind, so we rely on a general sense that this kind of work has historically provided almost exclusively benefits. As well as vague intuitions about how very advanced theories spawned from this direction could help Threat modeling / Benchmarking / Prevention, inform implementable Cooperative AI, and also clarify more speculative proposals like Acausal trade.

There is also an obvious sense in which this better understanding of intelligence in general could be useful for the construction of agents, although we think we are very far from this possibility and it's almost certainly not where most of the value of this research resides.

The tools developed for our purposes are also interesting theoretical advancements on logical uncertainty (and especially probability-based agents), and in particular we think a standardized understanding of inductive extrapolation of beliefs can prove useful in other settings, and could be interestingly central to Actor-Principal theories of Alignment.

Finally, a short assessment of possible backfire risks concluded (as was superficially predictable) that this kind of work is very far from anything that would push capabilities any time soon, and other paths to negative consequences are even more speculative. Although, again, the consequences of foundational theoretical work are highly unpredictable, and we just mostly rely on a vague sense of consequences being strongly skewed towards positivity thanks to value of information.

2 Desiderata

We are looking for a computable agent (an algorithm implementing Updateless Decision Theory) that behaves a certain way when facing environments. We'll use the symbol $\mathbf{A}$ for the agent, and $\mathbf{\Omega}$ for the environment.[3]

The main complication is in pinning down what "behaves a certain way" means here. We have hand-wavy intuitions about which kind of reasoning we want the agent to implement, and specific recommendations like paying up in **Logical counterfactual mugging** (assuming indeed the digits of π are uniformly pseudorandom), but we don't have a general procedure to, given a situation, decide what we want the agent to do. Finding this procedure ($\mathbf{A}$) is the main problem we tackle in this work. The complications it presents are related to the ill-definedness of logical uncertainty and logical counterfactuals (more on this in Sect. 3.2 and Appendix D).

In this section we present informally the hand-wavy intuitions we want $\mathbf{A}$ to satisfy. As discussed in Sect. 5, not all of these are attained by our construction, and in fact some seem impossible to attain jointly.

The first, preliminary desiderata is just a roundup of specific recommendations that serve as motivating examples.

[3]While in the examples above $\mathbf{\Omega}$ is presented as a perfect predictor for simplicity of exposition, of course that's really a stand-in for any statistical pattern in reality that $\mathbf{A}$ can exploit, including probabilistic ones.

Desiderata 0 (Solves motivating examples)**.** **A** *satisfactorily solves some concrete decision problems, including*

1. *Empirical counterfactual mugging*
2. *Logical counterfactual mugging*
3. *Logical counterfactual mugging with math homework (see below)*
4. *Parfit's Hitchhiker (see below)*

   ...

Of course, given the non-existence of objective metrics for comparing decision theories (such metrics are already decision-theory-laden), there is room for normative disagreement about which actions are *satisfactory* in these examples. Here and below, we generally pump our normative intuitions in the direction of eradicating dynamic instability: "if other decision theories prefer precommiting when they have the chance (because they recognize the outcome of precommiting as globally optimal), it would be better if we could naturally achieve the same without the need for precommitments".

The not yet introduced motivating examples are the following:

> **Logical counterfactual mugging with math homework**
>
> Same as **Logical counterfactual mugging**, but now, to perform the action of paying up (e.g. a bank transfer), **A** has to tell $\mathbf{\Omega}$ the correct millionth digit of π.

This variation gets at the following: **A**'s updateless behavior should be motivated by instrumental considerations about global optimality, not about a lack of factual knowledge (or an impossibility to obtain it). It would be easy for a dumb agent to perform correctly in **Logical counterfactual mugging** by chance, just because it has no computational power to actually compute the millionth digit of π before selecting which policy to follow (and $\mathbf{\Omega}$ knows this). But we want our agent to exploit these situations even when it has that factual knowledge, or could obtain it. We want **A** to use its knowledge strategically and in a fine-grained way, without fleeing away from information sources (which could provide upsides in many situations) due to a fear of updating. We want it to receive arbitrarily much information, and integrate it strategically into its decision procedure.

> **Parfit's Hitchhiker**
>
> You (**A**) are stranded in the desert. The perfect predictor $\mathbf{\Omega}$ will save you (something you value at \$1M) iff it predicts you will pay \$100 (that you can extract from your bank account through an ATM) when you get to the safe city.

This problem is functionally equivalent to Transparent Newcomb's, that is, Newcomb's problem with transparent boxes. Again, CDT and EDT want to precommit to pay up, but have no incentive to pay up once saved. The formal definition of this problem already enters the shaky ground of logical counterfactuals, because since **A** is a finitely described algorithm (and $\mathbf{\Omega}$ will have to be as well), there is a mathematically "determined" fact of the matter as to whether $\mathbf{\Omega}$, when fed **A**'s code, decides to Save. If it decides to Not save, the "simulation" trying to determine whether **A** would pay up if saved is a logical counterfactual.

The solving of these and other specific problems will be implied by the following, more general desiderata, that extrapolate the intuitions obtained from those specific examples in the natural directions. Some of the following desiderata are not independent of each other either.

Desiderata 1 (Dynamic stability)**.** **A** *doesn't self-rewrite. Equivalently,* **A** *doesn't need precommitments: it can always just implement the action it finds globally optimal.* **A** *doesn't strictly prefer a precommitment over another action.*

For example, the outcome of **Logical counterfactual mugging** stays the same regardless of whether we let the precursor agent precommit to paying up. This property has also been called Tiling in the literature.

Desiderata 2 (Updating as much as is instrumentally optimal)**.** **A** *carefully trades off the value obtained from updating on a piece of information against the possible negative (non-cartesian) consequences of making its decision depend on that piece of information.*

As a consequence, when **A** believes making its decision depend on a piece of information doesn't worsen its bargaining position, or have any negative consequences, **A** integrates that piece of information into its decision. In particular, when **A** has no reason to suspect its environment contains such non-cartesian channels of influence, it is completely updateful.[4]

Desiderata 3 (Eventual updating)**.** **A** *eventually (if maybe slowly) updates on all facts relevant to its situation.*

A central worry for Updateless Decision Theory is that it will "act silly in the limit": It has embraced a prior (the beliefs of the predecessor agent), and refused to update past it, forever committed to some sub-optimal course of action (if it had any wrong factual information). Since any prior will realistically contain only a finite amount of information and considerations, we want a way to prevent such dogmatism. Desiderata 2 and Desiderata 3 get at this, and according to them we'll need to integrate the strategic finite-time benefits of Updatelessness with eventual learning.

Desiderata 4 (No existential regret)**.** *In <u>fair</u> decision trees,* **A** *wouldn't prefer to have been another algorithm, that is, doesn't believe that another algorithm obtains higher reward in this decision tree.*

This might seem equivalent to dynamic stability, but there's a subtle difference. For dynamic stability, precommitments pertain to fixing an action sequence for the future, so that "self-rewriting" talks about the logical counterfactual in which **A** chooses to do that (when given as observations a decision tree allowing for precommitment). Instrumentality, on the other hand, deals with the notion of "wishing I had been a different algorithm" (which is different from "wanting to rewrite to that algorithm right now"), which is completely mathematically defined (and not a logical counterfactual) as "how much reward that other algorithm obtains when plugged into this same decision tree".

Intuitively, this happens because $\mathbf{\Omega}$ can look not only at **A**'s actions, but also its code. That's why Desiderata 4 can't be demanded in general. Consider, for example, for any **A**, the decision tree that punishes having **A**'s algorithm. Or also, the decision tree that punishes having the right belief about the truth of φ, so that **A** (regardless of what it believes about φ) would always rather believe the opposite.

This is not surprising: It is well-known that the existence of certain traps in the environment make some kinds of intuitively desirable learning or behavior impossible, and so can only be regarded as unfair [8]. Here, we are only dealing with a different kind of trap, and we will want to find a class of <u>fair</u> decision trees as broad as possible. Unfortunately, we will see in Sect. 5 that logical uncertainty makes a fully satisfactory definition unlikely.

Desiderata 5 (Good performance on disconnected sequences)**.** *When facing an infinite sequence of finite, <u>disconnected</u> trees (that is, the outcomes of one tree don't affect the others),* **A** *eventually obtains the best possible reward.*

[4] We will formalize this last case as **A** behaving exactly like Logical Inductor (Evidential) Decision Theory, see Sect. 3.1.1.

That is, infinite sequences of finite trees should be fair. Intuitively, regardless of how "unfair" the individual trees are, **A** will always have the opportunity to start afresh at the next tree, so it shall eventually learn and implement the best global policy in each finite tree (without having lost infinite reward). Again, though, logical uncertainty will complicate defining disconnected.

Desiderata 6 (Good average performance)**.** **A** *has optimal average reward in the class of all decision trees (or fair decision trees, or other classes).*

In Sect. 5.4 we discuss possible definitions for average.

Desiderata 7 (LIDT rewrites to UDT)**.** *An algorithm* **L** *implementing Logical Inductor (Evidential) Decision Theory (see Sect. 3.1.1), in a fair decision tree, and given infinitely often an opportunity to rewrite (for arbitrarily long finite stretches) to an implementation of UDT, learns to rewrite.*

As explained in Sect. 3.1.1, using Logical Inductors we can build the decision theory LIDT, which can be interpreted as implementing completely updateful instrumental reasoning. Since UDT can take advantage of globally (including counterfactually) optimal planning in a way that **L** cannot, under some weak assumptions **L** should notice the pattern that it always obtains higher reward when rewriting to UDT.

This desiderata ensures that a reasoner optimizing purely for epistemics (a Logical Inductor) arrives at the conclusion that our agent performs well. It's a priori not clear, though, what the implementation of UDT should be here, since we need to translate the knowledge that **L** has amassed into the UDT agent that will take the reigns for the finite stretch. This is explored in **??**.

Desiderata 8 (Coherent beliefs)**.** **A***'s beliefs satisfy some coherence properties:*

1. *Propositional Coherence: Since we deal with a probabilistic distribution, we mean properties like* $\mathbb{P}(\varphi \wedge \phi) \leq \mathbb{P}(\varphi)$*, with quantified logical sentences* φ, ϕ *playing the role of Boolean atoms, as in [2].*
2. *Logically Uncertain Variable (LUV) Coherence: A LUV is a real value defined by a logical formula* $\varphi(x)$ *(as in [2]). We want beliefs to coherently represent their interrelationships, so that their expected values behave as they should. For example, if we have a belief that* $x = 2 \cdot y$*, and that* $y \geq 3$*, we should also have a belief that* $x \geq 2 \cdot 3$*.*
3. *Reflection principle:* **A** *will have different credences on the possible future evolutions of its beliefs, and these should adequately marginalize to its current beliefs.*
4. *Epistemic self-trust: When* **A** *learns that it will come, in the future, to believe* φ *with probability* p*, it should automatically believe* φ *with probability* p*.*
5. *Coherence between small and large beliefs: At each finite time,* **A** *will have gathered only a finite amount of information. Thus, it will have sensible opinions on small-sized logical sentences, but not necessarily on large-sized ones. Nonetheless, we want to propagate its knowledge so that large opinions about the future are informed by the sensible small beliefs.*
6. *No added information in large beliefs: And also, we want a neutral way to perform the above extrapolation, so that we don't add arbitrary information that could play to* **A***'s strategic disadvantage.*

Notice first we are (here and also in previous Desiderata) talking about the beliefs of **A**. If we were to deal with an arbitrary algorithm **A**, we would need to specify a way to extract beliefs from observed behavior. But here that won't be necessary, since below it will be natural to directly construct **A** with an

explicit representation of beliefs, a probability distribution $\mathbb{P}$.

These (and some other) coherence properties do not only seem common-sensically positive, but will also play a crucial role in letting our agent decide future policy. In particular, property *5. Coherence between small and large beliefs* can be understood as a way for a small bounded agent to inform the actions of a big bounded agent (its more knowledgeable future self). This property also resonates with a Principal-Agent setup, in which a small bounded agent (a human) wants to instill its prior into a big bounded agent (an AI).

3 Framework

3.1 Preliminaries on Logical Inductors

Garrabrant's Logical Inductors are a natural model of logically uncertain reasoning and learning. Start by considering a sequence $\mathbb{Q}$ of (finite-support) probability distributions $\mathbb{Q}_n$ over logical sentences. That is, for every n and every logical sentence (in a certain language) φ, we have $\mathbb{Q}_n(\varphi) \in [0,1]$. The variable n is interpreted as days, and with the passing of days the reasoner comes to better knowledge and predictions about the truth or falsehood of logical sentences, by expending computational resources.

Each day's probabilities represent this accumulated knowledge. For example, once the reasoner learns the first digit of π, she will set the probability of its parity being odd to near certainty. Or, when it arrives at the strong mathematical conjecture that digits of π are uniformly pseudorandom, she will set sentences of the form "the mth digit of π is even" (for very large m that it cannot yet compute) to 0.5.
But there's another way to interpret these probabilities. Consider $\mathbb{Q}_n(\varphi)$ as the *price* (on day n) of a *share* in φ. Say this share returns 1\$ when φ is proven true (and nothing else in any other occasion). Then indeed, a reasoner's probability of φ will be exactly the price she would pay for acquiring such a share (if she wants to maximize expected wealth).

We can consider *traders* as programs that indeed, each day, make some movements in the market with prices $\mathbb{Q}_n$, buying and selling a finite amount of shares. We say a sequence $\mathbb{Q}$ as above is a Logical Inductor if it cannot be exploited by any efficiently computable[5] trader. This intuitively means that the trader can make infinite returns (in some possible world) by just investing a finite amount of wealth. It also turns out that we can construct computable Logical Inductors, by running a market of efficiently computable traders[6]. For actual mathematical definitions and the many technical details, we direct the reader to [2].

We are interested in the above because the Logical Inductor criterion implies many other desirable properties of the sequence $\mathbb{Q}$, which correspond to intuitively correct modes of logically uncertain reasoning. Some of the properties we'll most make use of in this work are:

- **Learning of (efficiently describable) statistical patterns:** For example, if indeed "the nth digit of π is even" is true for exactly half of the ns (in the limit), $\mathbb{Q}$ will eventually learn this pattern.

[5] We usually think of efficiently computable as meaning polynomial-time computable, but the definition of Logical Induction is parametric on this complexity class, so we can vary it at will.

[6] Although unfortunately computing Logical Inductors as constructed is extremely inefficient, that is, much more computationally costly than the efficiently computable class of traders we choose.

- **Expectations:** From a distribution $\mathbb{Q}_n$, we can define an expectation operator $\mathbb{E}_n$, that represents $\mathbb{Q}_n$'s guesses about the (real-numbered) values of certain logical variables, defined by some formula $\varphi(x)$ (these are called Logically Uncertain Variables in the original paper). This $\mathbb{E}$ converges towards many desirable coherent behaviors, motivating its use as an expectation.
- **Self-reference:** $\mathbb{Q}$ learns to correctly reason about self-referential sentences (while resisting paradoxes), like for example the liar sentence $\chi_n^p :\equiv \left(\mathbb{Q}_n(\chi_n^p) < p\right)$.
- **Self-trust:** $\mathbb{Q}$ learns to trust its own future epistemic states. For example, when $\mathbb{Q}_n$ is conditioned[7] on a sentence referencing the future like "$\mathbb{Q}_{n+k}(\varphi) = p$", it will itself now grant probability approximately p to φ.

3.1.1 Logical Inductor Decision Theory

As anticipated in Desiderata 7, we can also use Logical Inductors to define a decision theory natively dealing with logical uncertainty and embeddedness. We simply introduce new syntactic variables U_n and A_n to the language, representing the utility obtained, and the action taken by the Logical Inductor, in round n. In each round n, we update the Logical Inductor on the past observations (or just let it run for one more step if there are no new observations, which already improves its mathematical opinions). To let the Inductor decide between actions, we simply use the well-behaved expectations $\mathbb{E}_n$ extracted from the current probability distribution $\mathbb{Q}_n$ to assess them, and choose the one that looks best to $\mathbb{Q}_n$:

$$A_n := \text{argmax}_a \mathbb{E}_n(U|A_n = a)$$

That is, we take logical counterfactuals conditioning on taking any possible action ($A_n = a$), and choose the one that leads to the highest U[8], according to $\mathbb{Q}_n$.

Clearly this decision theory integrates logical uncertainty, since the Logical Inductor is a bounded agent improving its guesses about logical truths. It's also embedded in the sense that it can reason about itself, its own beliefs, and their possible effects on the environment (thanks to how Logical Inductors deal with self-reference).

We also want to add ε-exploration to this algorithm. Otherwise, a Logical Inductor whose "prior" (the initial wealth of its traders[9], which play a role similar to hypotheses) is biased against taking some action will never take it, and miss out on possibly surprising information (for more discussion, see [1]).[10]

[7] Conditioning $\mathbb{Q}$ (on a single sentence, or an efficiently computable sequence of sentences) preserves the Logical Inductor criterion.

[8] This U need not be just U_n: we can maximize any Logically Uncertain Variable. It can be the total sum of utilities (when this is finite), or time-penalized utility in an infinite tree $U_{\geq n} := U_n + \delta \cdot U_{n+1} + \delta^2 \cdot U_{n+2} + \ldots$ (with δ low enough that this amount is finite). Notice the latter changes with round number, as happens in many infinite-horizon Reinforcement Learning setups. One can also do the same with a dynamic δ that tends to 1 as rounds pass, thus ever-decreasing the agent's present bias.

[9] Strictly speaking, in the construction of [2] we don't directly allocate any initial wealth to the traders. The actual arbitrary decision we're referencing here is the choice of Universal Turing Machine used to enumerate the traders, which will arbitrarily favor some of them as simpler.

[10] A natural way to introduce this ε-exploration is through a sequence of self-referential sentences: $S_n :\equiv \mathbb{Q}_n(S_n) < \varepsilon$. We hard-code that, when such a sentence is true, our agent takes an action at random. This amounts to "when I'm too certain that I won't ε-explore, I'm forced to ε-explore". By diagonalizing against the Logical Inductor, we ensure it can't predict its ε-exploration rounds, and nonetheless stronger predictors can do it (it is desirable to account for this embedded possibility).

Conditioning as above is of course the EDT way of computing expected utility, so we call this Logical Inductor (Evidential) Decision Theory.[11] It is also completely updateful, in the sense that it always uses the most updated possible Logical Inductor state to arrive at decisions. Many of our work can be understood as finding a formalization of UDT by altering LI(E)DT.

3.2 Implementing logical counterfactuals

Empirical counterfactuals are perfectly well-defined: since the outcomes of empirical coins (or empirical observations in general) are a syntactic feature external to the decision tree, interpreted as inputs to whichever algorithms we use to compute the outcome of the situation ($\mathbf{A}$ and $\mathbf{\Omega}$), they're nothing more than a regular computation. Logical counterfactuals, on the contrary, alter "what regular computation even is", bringing into question whichever computations we use to decide the outcome.

Say $\mathbf{A}$ faces a **Logical counterfactual mugging**, and it turns out its logical coin φ is false. We can ask, "what would have happened if φ were true?" But there's an obvious intuitive sense in which there's no fact of the matter about the answer to this question. What would have happened if 2+2=3? That's just a possibility that our mathematical framework doesn't account for.

Realistically speaking, any definition of logical counterfactuals (that a predictor like $\mathbf{\Omega}$ can use to take decisions) will just be a computation we can perform on the code of the agent. For example, $\mathbf{\Omega}$ might look inside the agent's brain for a node labeled "truth value of φ", and when taking the counterfactual where φ is false, it just runs the agent algorithm with a causal intervention that fixes that node to 0. The problem with this and similar approaches is of course the mind dependence: What if an agent doesn't have such a node? $\mathbf{\Omega}$ will have to make a decision about how to compute the counterfactual in that situation as well. Since we don't have a good concept for "which computations (out of all the space of computations) are related to φ being true" (it's not even clear whether such an objective notion exists), such approaches will devolve into arbitrary mind-dependence. And we actually want to assess how all possible minds ($\mathbf{A}$s) fare in these situations.

We want some more natural definition that, at least in the limit, and in some contexts, eradicates mind-dependence. And Logical Inductors can help. Consider a Logical Inductor $\mathbb{Q}_n$ in a state advanced enough that it has some good understanding of basic math and how simple algorithms work, but not yet advanced enough to believe with certainty that φ is true. Consider its conditionals on the logical counterfactual, of the form $\mathbb{Q}_n(\mathbf{A}(\varphi) = a|\neg\varphi)$. For example, maybe $\mathbb{Q}_n$ has the strong opinion (due to past observations, or literally reading its code) that $\mathbf{A}$ never plays a when the sentence φ starts with a quantifier. Or maybe, that it just checks whether the sentence is true and acts accordingly, and so φ's truth value will be a clear input to its decision.

We claim that these conditionals will be in a sense "pretty reasonable". Through them, the traders of $\mathbb{Q}$ are just considering some syntactic properties of $\mathbf{A}$, φ and a that have been useful in the past to predict truth, and extrapolating them to the current situation. They are "approximating" the many detailed interactions between logical sentences, extrapolating ever-finer-patterns that give a more real picture of what

[11] The alternative Logical Inductor (Causal) Decision Theory is obtained by conditioning on the fact "I perform this action during ε-exploration", instead of "I decide to perform this action" (thus screening off any information the action provides about the agent's cognition). We don't deal with this alternative in this work.

would happen were φ to be false. Of course, if this picture got too fine, too accurate, $\mathbb{Q}_n$ would just notice that φ cannot be false. It would start noticing more and more inconsistencies with the situation that will inform its opinions. But while φ is still uncertain, the conditional will remain a good coarse-grained picture of what would happen.[12]

So consider the following proposal: $\mathbf{\Omega}$ uses a Logical Inductor $\mathbb{O}$ throughout, feeding it as its deductive process (that is, observations, updates, see [2]) the actions taken by $\mathbf{A}$, and also those decisions arrived at by $\mathbf{\Omega}$ (some of these decisions will depend on $\mathbb{O}$, so these sentences become self-referential). When we have to assess the likelihood of B given a logical counterfactual C, $\mathbf{\Omega}$ takes the latest state $\mathbb{O}_n$ that still hadn't decided the truth of C, and uses $\mathbb{O}_n(B|C)$ as its guess.[13] Notice $\mathbb{O}$ is not used as an LI(E)DT agent, it's just an observer we employ for its good opinions on math.

The following definition characterizes the situations we can construct with this computational machinery.

Definition 1. A *decision tree* $\bar{T}$ is a rooted countable tree (w.l.o.g. of degree two) such that

- Every node is labeled as a decision by either $\mathbf{A}$ or $\mathbf{\Omega}$.
- Of course, every $\mathbf{A}$-decision is decided by $\mathbf{A}$'s output, on input past observations and the whole decision tree[14].
- Each $\mathbf{\Omega}$-decision is taken in one of the following three ways:
  - *Empirical coin*: A number in $[0,1]$ is annotated in the node, representing the probability of Heads (although w.l.o.g. all coins can be chosen fair). The coin represents a syntactically introduced external random bit completely unpredictable to $\mathbf{A}$ or $\mathbf{\Omega}$.
  - *Logical coin*: A decidable sentence is annotated in the node[15]. $\mathbf{\Omega}$'s decision is univocally determined by the truth or falsity of this sentence.
  - *Entanglement*: $\mathbf{\Omega}$ uses a certain Logical Inductor $\mathbb{O}$ over the deductive process $\bar{D}$, where D_n equals either $\{\mathbf{A}(\bar{o}_{<n}) = \underline{\mathbf{A}(\bar{o}_{<n})}\}$[16] (if last node was an $\mathbf{A}$-decision) or $\{\mathbf{\Omega}(\bar{o}_{<n}) = \underline{\mathbf{\Omega}(\bar{o}_{<n})}\}$ (if it was an $\mathbf{\Omega}$-decision, either a coin or an entanglement). $\mathbf{\Omega}$ consults

$$\mathbb{O}_i((\neg)\mathbf{A}(\bar{o}) = a \mid \varphi)$$

[12] A different, more mechanistic explanation of why we need $\mathbb{Q}$ not to have decided φ yet for this to work is the following. When a conditional (like $\neg\varphi$) goes to low enough probability, successful traders stop devoting their movements to it. Or, better said, there's a general incentive for all traders to "fry bigger fish" (it seems likely that their bets on the conditional will not report them any benefits, since the conditional won't be resolved), and so the traders that continue betting on the conditional (instead of doing other things) generally keep losing share in the market, and are not as strongly selected for good performance. Thus, the opinions inside the conditional will not keep getting refined and updated, forgotten by most traders and not selected for any proxy of accuracy. It is possible for some traders, with enough wealth coming from other "bigger fish", to continue "taking care" of this conditional. But "on average" (across all weighted traders), they will end up less and less attended than the real conditional.

[13] Defining "hasn't decided the truth of C" in general is problematic. We could interpret it as "the probability is more than ε away from 0 or 1". But then what about a probability that sits at 0.9ε for a long time, but only gets decisively decided much later? In all that time $\mathbb{Q}$ can have continued to gather more syntactical insights, that would be useful to include in the conditional. Probably choosing a decreasing ε, or something that takes into account a general proxy for convergence times, will be a natural solution for most situations. In any event, we need not worry about this yet, since we can just let each decision tree choose this (as in Definition 1).

[14] Of course, if the tree is infinite, the finite algorithm $\mathbf{A}$ will only be able to consult a finite part of the tree before halting, which will be treated as an infinite oracle tape.

[15] Equivalently, we can use halting computations instead of decidable sentences.

[16] $\underline{f(x)}$ is notation for the explicit numeral representing the actual value of $f(x)$. Thus, we are just updating $\mathbb{O}$ on those values.

for a certain observation history $\bar{o}$, action a, logical sentence φ and $i \in \mathbb{N}$ smaller or equal to the present round number n. $(\neg)$ is notation for a sentence being possibly (but not necessarily) negated. We could also accept arbitrary Boolean combinations of the $\mathbf{A}(\bar{o}) = a$ prime formulas. $\mathbf{\Omega}$'s decision is a computable function of the above rational value. Both an algorithm for this function and the expression $\mathbb{O}_i((\neg)\mathbf{A}(\bar{o}) = a \mid \varphi)$ are annotated in the node.

- Every node is labeled with a (finite, rational) utility U, that $\mathbf{A}$ obtains when that node is reached.

Example 1 (Formalization of **Logical counterfactual muggings**)**.** Say Generator() is a fixed halting program such that Generator(n)=φ_n is a decidable logical sentence for every n. In round $2n$ of the decision tree (independently of what has happened before), $\mathbf{\Omega}$ decides φ_n (this is an $\mathbf{\Omega}$-decision, more concretely a Logical coin). If it's false, $\mathbf{A}$ is led to an $\mathbf{A}$-decision where it can decide whether to go for $U_{2n+2} = -100$ "Pay", or $U_{2n+2} = 0$ "Not pay" (these utilities are annotated in the subsequent nodes). If it's true, $\mathbf{\Omega}$ uses its Logical Inductor $\mathbb{O}$, consults $\mathbb{O}_{2n+1}(\mathbf{A}(\bar{o}) = \text{Pay}|\neg\varphi_n)$ (this is an Entanglement), and leads $\mathbf{A}$ to a node with $U_{n+2} = \$1000 \cdot \mathbb{O}_n(\mathbf{A}(\bar{o}) = \text{Pay}|\neg\varphi_n)$[17]. We could instead define $U_{n+1} = \$1000$ iff $\mathbb{O}_{2n+1}(\mathbf{A}(\bar{o}) = \text{Pay}|\neg\varphi_n) \geq p_n$, for a certain p_n. Assume also that $\mathbb{O}$ is slow enough not to decide φ_n by round $2n$, so that it still has good guesses about the logical counterfactual. For example, this is satisfied if the Logical Inductor is polynomial, and φ_n is the parity of the 2^nth digit of π (and the strong conjecture that there's no efficient way to compute the parity of digits of π is true).

Example 2 (Formalization of **Parfit's Hitchhikers**)**.** Two different rounds alternate infinitely:

1. In round n even, $\mathbf{\Omega}$ calculates $\mathbb{O}_n(\text{“}\mathbf{A}(\bar{o}_{<n+1}) = \text{Pay”} \mid \text{“}\mathbf{\Omega}(\bar{o}_{<n}) = \text{Save”})$ and plays Save iff it's $> p$.
2. In round n odd, if $\mathbf{\Omega}$ played Save in the previous round $\mathbf{A}$ receives 1.000\$, and then chooses whether to Pay 100\$. If $\mathbf{\Omega}$ played Not save, nothing happens at this node.

This presents some more complex self-reference. Indeed, for advanced enough Inductor states, we have

$$\mathbb{O}_n(\text{“}\mathbf{A}(\bar{o}_{<n+1}) = \text{Pay”} \mid \text{“}\mathbf{\Omega}(\bar{o}_{<n}) = \text{Save”}) = \mathbb{O}_n(\text{“}\mathbf{A}(\bar{o}_{<n+1}) = \text{Pay”} \mid S_n > p) =: S_n$$

$\mathbf{A}$'s behavior will nonetheless affect the resolution of this self-reference. For example, if $\mathbf{A}$ does something very simple, like actually Paying a fraction s of the time (by consulting a hard Pseudorandom Sequence), then some trader in $\mathbb{O}$ will arbitrate this, and this belief will be held correct (even when $\mathbb{O}_n(S_n > p)$ limits to 0, if $s \leq p$).

Definition 2 (Computable tree)**.** We say a decision tree $\bar{T}$ is *computable* when there exists a halting program Round() that, given a description of past observations and the last $\mathbf{A}$-decision or $\mathbf{\Omega}$-decision taken, computes the description of the next node of the tree: Round($\bar{o}_{<n}, a) = o_n$. In that case, we can choose to include the observations about Round() (of the form Round($\bar{o}_{<n}, a) = o_n$) in $\mathbb{O}$'s deductive process.

In fact, we can also include other arbitrary mathematical facts in $\mathbb{O}$'s deductive process. For example, $\mathbf{\Omega}$ might want to feed $\mathbb{O}$ a "basic set" of mathematical facts before starting to run it to make decisions.

As mentioned in Definition 1, $\mathbf{A}$ always has complete access to the decision tree. Translating this into a more realistic situation in which $\mathbf{A}$ is uncertain about the shape of the environment it inhabits is straightforward: intuitively, we will just implement a probability distribution over decision trees (and fix any preferred way to make inferences about this knowledge), and perform Expected Value Maximization also over that. In fact, we can implement any such situation ("simulate any uncertainty over decision trees") as a single decision tree with the right structure. That's also why we can accommodate for $\mathbf{\Omega}$ to look at our beliefs about this distribution over decision trees, and embeddedness is not lost.

[17] In our formalism, this requires countably infinitely many next nodes available, one for each rational value.

4 Building a naive Updateless agent

4.1 The Plan

We want to build an algorithm **A** that satisfactorily navigates decision trees like the above (Sect. 3.2), especially satisfying the intuitions behind our desiderata (Sect. 2).

We want to use an infinite prior $\mathbb{P}$ to decide actions, according to their effects on the expected value of utility U. That is, we'll want to set

$$A(\bar{o}) := \text{argmax}_a \mathbb{E}^{\mathbb{P}}(U|A(\bar{o} = a)$$

This is just implementing the basic idea behind UDT: a fixed agent (the prior) at "logical time 0" deciding all future policy. We can do this with any infinite prior $\mathbb{P}$ (regardless of how we've obtained it), and call this agent $\mathbf{A}_{\mathbb{P}}$.

But we'll be especially interested in a specific way to obtain $\mathbb{P}$. Given a decision tree, and a Logical Inductor $\mathbb{Q}$[18], the algorithm $\mathbf{A}_{\mathbb{Q}}$ will do the following:

1. Run $\mathbb{Q}$ for some time.
2. At a certain step k, take the current $\mathbb{Q}_k$, and extrapolate it into an infinite distribution $\mathbb{P}$. We call this "freezing the prior".
3. Use the prior $\mathbb{P}$ to decide actions, according to their effects on the expected value of utility U (see Appendix C for a discussion of this U). That is, set

   $$A(\bar{o}) := \text{argmax}_a \mathbb{E}^{\mathbb{P}}(U|A(\bar{o}) = a)$$

   This can include deciding to be updateful. In fact, we'll keep running (the unaltered version of) $\mathbb{Q}$ after state k, and an opinion of $\mathbb{P}$ can be of the form "in this situation, ask $\mathbb{Q}_{k+n}$ what to do".
   And in fact, it is natural to treat the future $\mathbb{Q}_{k+n}$ as the observation histories themselves that $\mathbb{P}$ reasons about (that were above represented by $\bar{o}$). Intuitively, the sudden appearance of a bomb in a decision node will be captured by some $\mathbb{Q}_{k+n}$ suddenly starting to believe that there is a bomb.
4. It might seem like we're done, we can just decide actions now. But in fact, we still have one step left, necessary for sensible behavior: Deciding what to update on. The meaning of this is discussed in

By Step 1 we can mean one of two things. We can we run $\mathbb{Q}$ on basic math before starting to take action sin the decision tree, and let it think about the decision tree, and then freeze the prior before having taken any action in the tree. Or instead we can directly use $\mathbb{Q}$ to decide actions for the first rounds (using LI(E)DT), while it receives observations about how the tree responds, and at some finite round freeze the prior. Here (and even more clearly in deciding how big of a k to choose) we already observe a trade-off between learning and strategic updatelessness: more information about the tree can be very useful, but it can make us update on some facts we'd have been better off being strategically updateless about (more discussion on Sect. 5).

[18] As mentioned in Footnote 9, the arbitrary choice of Logical Inductor just means the arbitrary choice of Universal Turing Machine used to enumerate traders inside the Logical Induction Algorithm, which is equivalent to a prior over hypotheses.

To ensure that the resulting agent has some properties necessary for sensible decision-making, we'll have to ensure that $\mathbb{Q}_k$ satisfies some of the coherence properties of Desiderata 8 (that's Sect. 4.2), and that they are preserved by the extrapolation into $\mathbb{P}$ (that's Sect. 4.3).

But why do we need these properties? A driving intuition is that, for $\mathbb{P}$ (generated from finite information) to have any chance at reasoning correctly about the future, it must have sensibly coherent opinions about how its own beliefs (that is, the $\mathbb{Q}_{k+n}$) might evolve in the future. This will allow it to reason with minimal correctness about when to be updateful, so that it's not stuck forever with bad guesses. And to achieve this, the properties of the *Reflection principle*, *Epistemic self-trust*, and *Coherence between small and large beliefs* in Desiderata 8 are central. And it turns out they are naturally obtained through the also desirable properties of *Propositional Coherence* and *LUV Coherence*.

4.2 Making Logical Inductors Coherent

4.2.1 Propositional Coherence

Logical Inductors are not Propositionally Coherent at finite times. It is possible for $\mathbb{Q}_k$ to satisfy $\mathbb{Q}_k(\varphi \wedge \phi) = 0.9$ and $\mathbb{Q}_k(\varphi) = 0.4$ at the same time, since after all they are different logical sentences, and traders can bet on them whichever way they like. It is true that traders will learn to arbitrate propositional coherence, since it's a true fact about logic that will bring them economic benefit, and as a consequence $\mathbb{Q}$ will be propositionally coherent in the limit.

But we're not dealing with the limit here: we're choosing a finite $\mathbb{Q}_k$. We'll modify Logical Inductors so that they are Propositionally Coherent at every finite stage. Here's how we do it:
Usually, traders bet independently on all different logical sentences, as if they were on different betting "tables". That makes it possible for a trader to bet incoherent things on different tables. To fix this, we put down all Propositionally Coherent possible worlds on a single betting table, and only let traders bet on them.
Say the Logical Inductor has only been thinking about the quantified sentences $\varphi_1, \ldots, \varphi_n$ (that is, no trader has placed any order outside of these). Then our Propositionally Coherent possible worlds will be all conjunctions of the form $(\neg)\varphi_1 \wedge \ldots \wedge (\neg)\varphi_n$. We ensure that their prices always sum to 1.
Whenever a trader wants to buy a share of a sentence, we just sell it a share in all possible worlds satisfying that sentence. This guarantees Propositional Incoherence, or losing money to it, is impossible. For example, if a trader wants to buy $\varphi \wedge \neg\varphi$, this is not satisfied in any possible worlds, so it will pay \$0 and obtain 0 shares. What if it wants to buy first one share of φ, and then one of $\neg\varphi$? These in total amount to buying a share in each possible world, and that's equivalent to buying nothing (you pay \$1, which is the sum of the price of all worlds, and get a guaranteed \$1, since exactly one of the worlds will turn out true). For more details and a proof, see [3].[19]

4.2.2 LUV Coherence

Given Propositional Coherence, there's an easy way to get LUV Coherence. For each formula $\varphi(x)$ that might define a LUV (say a LUV in $[0,1]$ for simplicity), fix a syntactic way of writing a formula $\varphi(n,k)$,

[19] As written in [3], Universal Inductors contain some further generalizations. In particular, each finite state $\mathbb{Q}_k$ has opinions on all logical sentences. But this is only attained by arbitrarily setting sentences not yet thought about to 0.5 probability. We'll ignore this, working with Logical Inductors with finite support, since we'll want an infinite extrapolation with better properties than this one.

whose truth represents "the LUV defined by $\varphi(x)$ belongs to $(\frac{k}{2^n}, \frac{k+1}{2^n}]$ (or $[\frac{0}{2^n}, \frac{1}{2^n}]$ when $k = 0$)". Fix also a formula φ_{LUV} representing "$\varphi(x)$ is indeed a LUV", "the LUVs represented by $\varphi(x)$ and $\phi(y)$ are equal", etc.
When a Propositionally Coherent possible world on the betting table believes φ_{LUV} (that is, φ_{LUV} is not negated in the conjunction), we start imposing coherence properties on the other sentences. For example, it is incoherent for a world to believe two LUVs equal, and also that they satisfy different $\varphi(n,k)$ sentences. And so we simply remove all such possible worlds from the betting table. Or equivalently, we set their price to 0 permanently (and of course their shares will never report benefits, since they are always false).

This has the obvious shortcoming that it only applies to the few syntactic representative sentences that we have chosen. That is, there will be other sentences that are "intuitively" just a trivial re-statement of "$\varphi(x)$ is indeed a LUV", and nonetheless we are not checking them, and so the Inductor might have silly opinions on them. But this is the best we can do, since there's no privileged way to determine what is "intuitively" a trivial restatement, versus a very different and very complicated sentence that nonetheless turns out to have the same truth value as "$\varphi(x)$ is indeed a LUV". We could only completely propagate this knowledge through all such correlations, and that would just be logical omniscience.
But we are okay with our solution because, as long as when assessing expectations we look at the syntactic representatives we have chosen, we can be sure they will be LUV Coherent. Yes, there might be knowledge from the Logical Inductor that hasn't yet been propagated into these syntactic representatives (and the other way around), but that will get better with time thanks to traders more thickly arbitrating. And, if some property is not yet arbitrated and so our syntactic representatives are missing some logical insight, that's just the same as the Logical Inductor taking a bit longer to discover that insight, and in this framework we are generally okay with this fallibility (that is, having no strict finite time guarantees).

4.2.3 Universal Instantiation and other properties

One property that will be especially important for our setup is Universal Instantiation. That is, $\forall x \varphi(x)$ and $\neg\varphi(\underline{i})$ being jointly incoherent.

Why is this property important? To ensure *Coherence between small and large beliefs*. Since only a finite amount of information has gone into the prior $\mathbb{P}$ (that is, $\mathbb{Q}_k$ only has finitely many opinions), we need a way to extrapolate from small to large opinions. For example, $\mathbb{Q}_k$ won't have explicit opinions on very far-off decision nodes, since the numbers describing them are just too big. But it will be possible for $\mathbb{Q}_k$ to have indirect opinions on them through quantified sentences. For example, $\mathbb{Q}_k$ might believe that "for any future observation, if it satisfies property $P()$, then I should take action a". When we extrapolate into $\mathbb{P}$, we will ensure that this gets accurately translated into the particularizations of the form "for this concrete observation, if it satisfies property $P()$, then I should take action a", so that actions can be evaluated for this concrete observation.

If $\mathbb{Q}_k$ didn't satisfy Universal Instantiation, we wouldn't be able to do this sensibly. Maybe we'd try to fix $\varphi(\underline{i})$ to a high probability (for some concrete i), because $\mathbb{Q}_k(\forall x \varphi(x))$ is high. But then we'd find that $\mathbb{Q}_k(\varphi(\underline{i}))$ already exists, and is low. So we'd have to trash out arbitrary knowledge from $\mathbb{Q}_k$.

Similarly to LUV Coherence, there's an easy way to ensure this property: removing from the betting table those possible worlds which don't satisfy this Coherence.

And similarly we can enforce many other superficial coherence properties if we desire so, like for example $\forall x(\varphi(x) \wedge \phi(x))$ having the same truth value in all possible worlds as $\forall x\varphi(x) \wedge \forall x\phi(x)$.

4.2.4 Reflection principle and Epistemic self-trust

We cannot just ban some worlds from the betting table and be done with it (as above), since for example, given φ is true, it's perfectly coherent to have $\mathbb{Q}_{k+n}(\varphi) = 0.4$, and also perfectly coherent to have $\mathbb{Q}_{k+n} = 0.8$ (even if, if we trust $\mathbb{Q}$'s epistemic evolution, the latter should a priori seem more likely). And nonetheless, if the price of $\varphi \wedge \mathbb{Q}_{k+n}(\varphi) = 0.4$ were too high, Reflection would break. So this is a global property of $\mathbb{Q}_k$, instead of a local property of single possible worlds (which play the role of Boolean truth assignments).

To obtain this, we only let traders bet on the meta-beliefs (beliefs about future beliefs) up to a finite future horizon, and from them (by marginalization) extract their object-level beliefs about basic logic. That is, the Reflection principle is broken because traders can bet independently on object-level beliefs (φ) and on future beliefs about them ($\mathbb{Q}_{k+n}(\varphi) = p$). If we only let them bet on the latter, and from those deduce by marginalization the object-level prices of sentences, we'll obtain marginalization by default. And we can still, as above, let traders bet on object-level sentences: betting on φ will just amount to betting on many worlds where $\mathbb{Q}_{k+n}$ is high.

To see this more clearly, consider the case where $n = 1$. Say our traders can only bet on (possible worlds which are conjunctions of possibly negated) sentences of the form "$\mathbb{Q}_{k+1}(\varphi) \in (\frac{m}{2^{k+1}}, \frac{m+1}{2^{k+1}}]$" (that is, instead of allowing arbitrary precision of the form "$= p$" for any rational p, we use a coarse-graining, that gets more precise with time). Then, when a trader wants to buy an object-level share of φ, we actually sell it the bundle with $\frac{m+0.5}{2^{k+1}}$ shares of "$\mathbb{Q}_{k+1}(\varphi) \in (\frac{m}{2^{k+1}}, \frac{m+1}{2^{k+1}}]$". That is, we distribute its wealth so that future worlds in which φ seems more probable are more likely (according to this change in probability). And finally, once all traders have placed their bets and the market prices are set, we can just define

$$\mathbb{Q}_k(\varphi) := \sum \frac{m+0.5}{2^{k+1}} \mathbb{Q}_k(\mathbb{Q}_{k+1}(\varphi) \in (\frac{m}{2^{k+1}}, \frac{m+1}{2^{k+1}}])$$

That is, *Reflection principle* by definition. And then, when there's a mixture of object-level and meta-level sentences, by definition, we also set

$$\mathbb{Q}_k(\varphi \wedge \mathbb{Q}_{k+1}(\varphi) \in (\frac{m}{2^{k+1}}, \frac{m+1}{2^{k+1}}]) := \frac{m+0.5}{2^{k+1}} \cdot \mathbb{Q}_k(\mathbb{Q}_{k+1}(\varphi) \in (\frac{m}{2^{k+1}}, \frac{m+1}{2^{k+1}}])$$

That is, *Epistemic self-trust* by definition.

And then, the Coherence properties of the sections above will be implemented in the object-level beliefs of $\mathbb{Q}_{k+1}$: any possible world we are considering (in which $\mathbb{Q}_{k+1}$ turns out to have this set of opinions) is one where $\mathbb{Q}_{k+1}$ is Coherent (that is, we eliminate those which are not from the betting table). And as a consequence, through marginalization, those Coherence properties will also be translated into our derived object-level beliefs.

We can do a similar setup for the general n case, but since now the meta-beliefs of future different stages (for example, $\mathbb{Q}_{k+1}$ and $\mathbb{Q}_{k+2}$) also need to marginalize between themselves, some more complicated

construction needs to be carried out. We basically consider only "bundles" of probabilities across future trajectories (that is, worlds are the opinions of all future stages on all sentences in our finite support) that correctly marginalize (that is, we take the "incoherent" trajectories, in the sense that they don't correctly marginalize, off the betting table). And we only let traders buy these bundles. As above, buy a share on an object-level belief will amount to buying shares on trajectories according as to how much they ratify that belief.
For this general case, there remain some technicalities to resolve, so it doesn't seem worthwhile for now to include all detail here.

4.3 Inductive extrapolation of beliefs

Now say we have a finite $\mathbb{Q}_k$, satisfying the above properties, and want to extrapolate it to an infinite $\mathbb{P}$ which preserves them, and correctly integrates and extrapolates all of $\mathbb{Q}_k$'s knowledge.

We will do this through an infinite induction. Say $\mathbb{Q}_k$ has opinions on the prime sentences $\varphi_1, \dots, \varphi_n$. That is, it has probabilities on all conjunctions $(\neg)\varphi_1 \wedge \dots \wedge (\neg)\varphi_n$.

As a succinct way to express some of the Coherence constraints we'll want to enforce in this induction, consider the two following axiom schema (infinitely many axioms):

1. $\forall \overline{m} \varphi(\overline{m}) \to \varphi(\bar{\underline{i}})$ for any φ and sequence of numerals $\bar{\underline{i}}$.
2. $\forall m_1 \forall m_2 (\varphi(m_1) \wedge \phi(m_2)) \leftrightarrow (\forall m_1 \varphi(m_1) \wedge \forall m_2 \phi(m_2))$

Call it Ax. It is propositionally consistent by being a fragment of First Order Logic. In fact, when implementing some of the Coherence properties on Logical Inductors above, what we've done is basically eliminating trades in worlds (finite conjunctions) that weren't consistent with Ax.[20]
Now consider the set of all logical sentences, minus all sentences of the form "Future($k+n$, φ, d)", plus the set of expressions "$\mathbb{Q}_{k+n}(\varphi)$". Fix an enumeration of this set such that

- $\varphi_1, \dots, \varphi_n$ is an initial segment
- φ always appears before Future($k+1$, φ, d), and Future($k+n$, φ, d) before Future($k+n+1$, φ, d), and Future($k+n$, φ, d) before Future($k+n$, φ, $d+1$).

As made explicit below, Future($k+n$, φ, d) stands for "all beliefs about the value of $\mathbb{Q}_{k+n}(\varphi)$, up to precision 2^d". And the second condition ensures the induction will be well-founded.

We inductively define $\mathbb{P}$ by setting conditional probabilities in the enumeration order, as follows (where C is an arbitrary conjunction, always in the order of enumeration):

- *Base case:* $\mathbb{P}((\neg)\varphi_1 \wedge \dots \wedge (\neg)\varphi_n) := \mathbb{Q}_k((\neg)\varphi_1 \wedge \dots \wedge (\neg)\varphi_n)$
- *Syntactic Coherence:* $\mathbb{P}(C \wedge \varphi) := \mathbb{P}(C)$ whenever $\mathrm{Ax} \cup \{C\} \vdash_{\texttt{Prop}} \varphi$, and $\mathbb{P}(C \wedge \varphi) := 0$ whenever $\mathrm{Ax} \cup \{C\} \vdash_{\texttt{Prop}} \neg\varphi$.
- *Future beliefs:* Whenever the expression Future($k+n$, φ, d) appears, we'll need to find, given all the $\mathbb{P}(C)$ we already have (an initial segment of the enumeration), a distribution for all the $\mathbb{P}(C \wedge \mathbb{Q}_{k+n}(\varphi) \in (\frac{m}{2^d}, \frac{m+1}{2^d}))$. We won't explicit the details here, but basically there's always such

[20] We could also implement here axioms representing LUV Coherence, although usually it's easier to treat general statements about LUVs (like inequalities) as, by definition, a more complex statement about digits of LUVs (and the latter sentences will already behave correctly due to Propositional Coherence).

a way to find this distribution so that the values of $\mathbb{Q}_{k+n}(\varphi)$ correctly marginalize to those of $\mathbb{Q}_{k+n-1}(\varphi)$ (or $\mathbb{P}(\varphi)$ if $n = 1$), when n has been increased, or that accordingly correspond to the previous more coarse-grained version of this distribution, when d has been increased. Importantly, we can also choose these distributions so that the $\mathbb{Q}_{k+n}(\varphi)$ satisfy the Coherence properties that $\mathbb{Q}$ (and also $\mathbb{P}$) do indeed satisfy. For example, it is always possible to ensure that if $\mathbb{Q}_{k+n}(\forall x\varphi(x)) \geq p$, then $\mathbb{Q}_{k+n}(\varphi(\underline{i})) \geq p$.[21]

- *Agnostic clause:* $\mathbb{P}(C \wedge \varphi) := \frac{1}{2}\mathbb{P}(C)$ whenever none of the above clauses are triggered.
- Of course, $\mathbb{P}(C \wedge \neg\varphi) := \mathbb{P}(C) - \mathbb{P}(C \wedge \varphi)$.

And given any formula φ, we consider all finite conjunctions C with φ's prime formulas as literals, and define $\mathbb{P}(\varphi) := \sum_{C \vdash_{\texttt{Prop}} \varphi} \mathbb{P}(C) = \sum_{\text{Ax} \cup \{C\} \vdash_{\texttt{Prop}} \varphi} \mathbb{P}(C)$.

Since $\mathbb{Q}_k$ was already Coherent in the necessary ways, it's immediate to prove by induction that $\mathbb{P}$ also is.

Also, note that deciding the predicate $\text{Ax} \cup \{C\} \vdash_{\texttt{Prop}} \varphi$ for finite C is decidable.

We get the nice equality

$$\mathbb{P}(\forall \overline{m}\varphi(\overline{m})) = \lim_{\overline{m}} \mathbb{P}(\bigwedge_{\overline{m}} \varphi(\bar{\underline{i}}))$$

Indeed, Ax ensures $\mathbb{P}(\forall \overline{m}\varphi(\overline{m})) \leq \lim_{\overline{m}} \mathbb{P}(\bigwedge_{\overline{m}} \varphi(\bar{\underline{i}}))$, since in all worlds (conjunctions) where the former is true, the latter also is. We also get the other inequality[22], because $\lim_{\overline{m}} \mathbb{P}(\neg\forall \overline{m}\varphi(\overline{m})) \wedge \bigwedge_{\overline{m}} \varphi(\bar{\underline{i}})) = 0$. Indeed, the latter could only fail if a finite and Ax-consistent $C \wedge \neg\forall \overline{m}\varphi(\overline{m})$ propositionally entailed infinitely many $\varphi(\bar{\underline{i}})$. This isn't possible with our Ax, because the only way to have this is through quantification. As a consequence, the *Agnostic clause* will be triggered infinite times, and the infinite multiplication by $1/2$ drives the limit to 0.

$\mathbb{P}$ also has the nice property that, when conditioned on information about future beliefs (or even the complete state description of a state $\mathbb{Q}_{k+n} = \mathscr{Q}$), it nicely extrapolates its finite beliefs into the infinite automatically (just as we have done with $\mathbb{Q}_k$). For example, conditioned on a high value of $\mathbb{Q}_{k+n}(\forall x\varphi(x))$, $\mathbb{P}$ will have high probability for that sentence as well, but then the extrapolation will ensure that $\mathbb{P}(\varphi(\underline{i}))$ is also high under this conditional.

On the note of arbitrary choices, most sentences' probabilities will still arbitrarily depend on enumeration order, and so be "off-base guesses". For example, this will be the case for any prime formula bigger than any formula in the support of $\mathbb{Q}_k$. We could arbitrarily fix an enumeration to make these guesses have arbitrary structure. It can be argued that this is not troubling, and inherent to bounded rationality: if our reasoner has only had a finite amount of time to think, then of course it will only have a finite amount

[21] Despite all these constraints, there are still always infinitely many distributions to choose from. So, arbitrarily, we always choose the one that's "as uniform as possible", in the sense of minimizing the total sum (across all C) of differences between conditional probabilities $\mathbb{P}(\mathbb{Q}_{k+n}(\varphi) \in (\frac{m}{2^d}, \frac{m+1}{2^d}) \mid C)$ (for one same C). This is meant to approximate maximum entropy, and so intuitively tries to satisfy *No added information in large beliefs* (see Desiderata 8). But ultimately, the effect of this or another choice on performance is not yet clear, so it remains a mathematically arbitrary choice.

[22] Intuitively, to a logician this inequality corresponds to $\mathbb{P}$ knowing that "we are in a standard model of the naturals", that is, all natural numbers are represented by a certain numeral. This cannot be expressed as a formula, and so we need to talk about limits.

of well-grounded information (despite there being infinitely many logical sentences). More specifically, these worries will be more satisfactorily dealt with when in Sect. 5 we ensure our reasoner keeps learning, so that it's never forever stuck with off-base guesses.

4.4 Deciding what to update on

The need for this final step is best demonstrated through an example. Say $\mathbb{Q}_k$ believes something like "whenever the decision node I'm in has this simple syntactic property, the best thing to do is this action". For example, it could believe "whenever $\mathbf{\Omega}$ asks me for the capital of France for the prize of \$1, and I say Paris, I obtain \$1". Here the simple syntactic property of the decision node is "$\mathbf{\Omega}$ asks me for the capital of France". This sentence will be formalized as

$$\forall \mathcal{Q}(P(\mathcal{Q}) \wedge \mathbf{A}(\mathcal{Q}) = \text{Paris} \rightarrow U = \$1)$$

where $P()$ is a description of the syntactic property.

Given any possible concrete future observation $\underline{\mathcal{Q}}$, Universal instantiation ensures that $\mathbb{P}$ also has a high probability on

$$P(\underline{\mathcal{Q}}) \wedge \mathbf{A}(\underline{\mathcal{Q}}) = \text{Paris} \rightarrow U = \$1$$

Say $\mathbf{A}$ finds itself having made that concrete observation. What we'd like to happen is for $\mathbb{P}$ to realize that $P(\underline{\mathcal{Q}})$, and thus the action $\mathbf{A}(\underline{\mathcal{Q}})$ will have the highest $\mathbb{E}^{\mathbb{P}}(U|A(\bar{o}) = a)$ and be chosen. But the problem is, $\mathbb{Q}_k$ might have had no opinions on whether $P(\underline{\mathcal{Q}})$ (even if $P()$ is intuitively very computationally cheap to check), if $\underline{\mathcal{Q}}$ is represented by a large enough number, that never appeared in its support. In fact, this will happen for all but a finite set of the $\underline{\mathcal{Q}}$. And so, $\mathbb{P}$ will not have either any strong opinions about $P(\underline{\mathcal{Q}})$.

It is thus unavoidable, if we want $\mathbb{Q}_k$'s small beliefs to count for something even in large scenarios, to choose some basic decidable properties about which we should be updateful when assessing policies. We can be updateful by just adding that knowledge to the conditional. In this case, if we want to be updateful about $P(\underline{\mathcal{Q}})$, we'd use arbitrary computation to decide its truth, and argmax $\mathbb{E}^{\mathbb{P}}(U|A(\bar{o}) = a \wedge P(\underline{\mathcal{Q}}))$ or $\mathbb{E}^{\mathbb{P}}(U|A(\bar{o}) = a \wedge \neg P(\underline{\mathcal{Q}}))$ accordingly (instead of $\mathbb{E}^{\mathbb{P}}(U|A(\bar{o} = a))$.

An intuitive first solution might be to fix a class of "simple enough properties" (that is, some complexity class), and be updateful about all of exactly those.[23] But actually, what we ultimately care about is performance, and in many situations "updating on simple properties" and "updating in an instrumentally optimal way" come apart. So it's more natural to ask $\mathbb{P}$ itself what it wants to update on. The following is a formalization of how to do that.

4.4.1 Formalization

Imagine that, when deciding what action to take on seeing observation $\mathcal{Q}$, instead of argmaxing a on $\mathbb{E}^{\mathbb{P}}(U|\mathbf{A}(\mathcal{Q}) = a)$ (not updating at all) or on $\mathbb{E}^{\mathbb{P}}(U|\mathbf{A}(\mathcal{Q}) = a \wedge \mathbb{Q}_m = \mathcal{Q})$ (updating completely), we

[23] This might involve conditioning on an infinite conjunction. We can do that with our infinite $\mathbb{P}$ when the limit we need to take converges. It's unclear if this limit always converges in this situation (or we can implement some small alteration to achieve that). But in any case we could also restrict the conditional to some accurately chosen finite conjunction, which also ensures computability.

choose some different sentence $\sigma(\mathscr{Q})$, and use $\mathbb{E}^{\mathbb{P}}(U|\mathbf{A}(\mathscr{Q}) = a \wedge \sigma(\Sigma, \mathscr{Q}))$. This $\sigma(\Sigma, \mathscr{Q})$ will be

$$\bigwedge_{\varphi \in \Sigma \cap \text{Support}(\mathscr{Q})} \mathbb{Q}_m(\varphi) = \underline{\mathscr{Q}(\varphi)}$$

for some set Σ of logical sentences. That is, we are updateful exactly about the sentences in Σ (and that, of course, $\mathscr{Q}$ has opinions on).

Our infinite prior $\mathbb{P}$ has some beliefs about the consequences of policies. Some are simple, like

$$\forall \mathscr{Q}(P(\mathscr{Q}) \wedge \mathbf{A}(\mathscr{Q}) = a \rightarrow \uparrow U)$$

But it also has some beliefs about more elaborate policies. Consider the policy we'd obtain if we were updateful exactly about the sentences in Σ, as formalized above. This policy can be described in our logical language, say by a logical formula $\text{Policy}(\Sigma, \mathscr{Q})$[24] (for infinite Σ, say they are described by a finite program that decides membership). A $\mathbb{P}$ obtained from an advanced enough $\mathbb{Q}_k$ will have opinions about these policies, for example of the form

$$(\forall \mathscr{Q}(\mathbf{A}(\mathscr{Q}) = \text{Policy}(\Sigma, \mathscr{Q})) \rightarrow \uparrow U$$

That is, "if I always update on exactly Σ, I'll do well".

So, once we have $\mathbb{P}$ (and before taking any further actions), we find the Σ that argmaxes

$$\mathbb{E}^{\mathbb{P}}(U|\forall \mathscr{Q}(\mathbf{A}(\mathscr{Q}) = \text{Policy}(\Sigma, \mathscr{Q})))$$

We fix this Σ, and proceed as usual, but adding the corresponding $\sigma(\Sigma, \mathscr{Q})$ to our conditionals.

This is still too coarse, since we just go with the same Σ set for eternity. But we can straightforwardly improve Policy(), and have a more complicated object instead of Σ, that allows for things like "before round n use Σ_1, and after Σ_2". More interestingly, we can accommodate for several meta-levels of deciding which information to update on: "first check whether $P(\mathscr{Q})$, and if that's the case check for $S(\mathscr{Q})$, otherwise don't check anything else". That is, intricately dealing with info-hazards.

In fact, there are some fancier ideas we could implement here, see Appendix E.

5 Properties, limitations and extensions

5.1 Desiderata revisited

It's clear that the behavior of $\mathbf{A}_{\mathbb{P}}$ depending on $\mathbb{P}$. Some priors will decide to be more updateful, others less, etc. This also happens for $\mathbf{A}_{\mathbb{Q}}$, although we might hope something more natural arises here, so that any $\mathbb{Q}$ run long enough converges on some basic decision-theoretic realizations (but here the learning vs updatelessness trade-off kicks in again).

As a consequence, many of our desiderata are met "when the prior contains the necessary knowledge". Indeed, the examples of **Desiderata 0** (Solves motivating examples) are immediately solved:

[24] We could also have Σ not be a variable, and instead have one formula $\text{Policy}_{\Sigma}()$ for each Σ. But maybe having it be an explicit parameter makes it easier for the Inductor to quickly notice some patterns.

1. Empirical counterfactual mugging is satisfactorily solved as long as $\mathbb{P}$ understands correctly the structure of the decision problem at hand (the probability of the coin, the payoffs, the decision node in which it takes place).
2. Same goes for Logical counterfactual mugging, as long as $\mathbb{P}$ has a "sensible" (according to our intuitions) probability for the logical coin. It's more interesting to consider "which kind of beliefs Inductors usually converge on", or limit performance when we ensure eventual updating (see Sect. 5.2).
3. Same goes for Logical counterfactual mugging with math homework: An advanced enough $\mathbb{Q}_k$ will understand the policy of extracting that knowledge and paying up is better in Expected Value.
4. Same goes for Parfit's Hitchhiker, although there are some worries that agents with more computational power than $\mathbf{\Omega}$ (that is, such that $\mathbb{Q}$ is a faster Logical Inductor than $\mathbb{O}$) might be able to do even better than a priori seems possible by diagonalizing against $\mathbb{O}$ (see Appendix F).

In all of these cases, the completely formal mathematical result is of the trivial form: "Assume the probabilities of $\mathbb{P}$ on these sentences are within these ranges. Then $\mathbf{A}_{\mathbb{P}}$ takes this action".

Desiderata 1 (Dynamic stability) is immediately met if we use a non-updating $\mathbb{P}$ (that is, we don't implement the mechanism in Sect. 4.4, or we do but $\mathbb{P}$ prefers being completely updateless). But if we introduce any kind of updating (even on the simplest kinds of syntactical properties), it becomes possible for one of the later, updated instantiations of the agent to prefer re-writing. There are some circumstances in which we can guarantee this doesn't happen (when the effects of these updates are small enough, or when the structure of the decision problems is strict enough that we hypothesize no incentives will exist even in the long run). But these seem like strict and unnatural assumptions. We seem to be facing the fundamental trade-off between learning and dynamic stability (see Sect. 5.2 for more).

Desiderata 2 (Updating as much as is instrumentally optimal) can be interpreted in two ways. If we interpret it subjectively from the perspective of the agent, it is trivially true: trading off the positive and negative consequences of any updating is literally what an Expected Value calculation does. If we interpret it objectively, as the agent actually updating as much as it is objectively optimal, the situation gets more complicated. In one sense, for every finite decision tree, a late enough $\mathbb{Q}_k$ will understand its structure, find the optimal policy given the knowledge it already has, and implement it (but of course, it might be now too late to be updateless about some information). In another sense, for any prior (or at least for any prior generated from a finite distribution), we'll be able to find a "non-adversarial" decision tree (see Sect. 5.3 for more) in which it fails either due to a lack of information (epistemics), or due to a lack of strategic updatelessness, that is, too much information (instrumentality). But this seems like a fundamental limitation of bounded rationality, and maybe we should consider average instead of worst-case metrics (see Sect. 5.4).

Desiderata 3 (Eventual updating) fails completely except on very specific priors which end up updating about everything. See Sect. 5.2 for an important discussion of learning versus updatelessness.

Desiderata 4 (No existential regret) will also fail, since the set of beliefs $\mathbb{Q}_k$ that we use as the seed for our agent hasn't been optimized at all for instrumentality (performing well on this decision tree), just for pure epistemic accuracy. If $\mathbf{\Omega}$ only looks at our actions (Appendix A), or things $\mathbb{P}$ can control freely without triggering any epistemic traps, the agent will just perform correctly this instrumentality. But if

Ω makes its decisions depend on the shape of $\mathbb{Q}_k$ or $\mathbb{P}$, or anything that might be correlated with it, $\mathbb{P}$ can perfectly know as a mathematical fact that a different prior (that had been optimized for instrumentality) would score better. The problem is, due to our confusion about logical uncertainty (or its subjectivity), we don't know how to draw a boundary isolating those "evil" kind of correlations. Or equivalently, we don't know how to define fair trees. **Desiderata 5** (Good performance on disconnected sequences) also suffers from this: What if a logical coin in one sequence is talking about our agent's behavior in another, as a mathematical sentence? And thus (although for us, from the "outside" of the decision tree, the coin is already decided), from the agent's perspective, altering its actions sub-optimally so as to change the truth value of that coin can be beneficial. We don't know how to prohibit this "evil" coins (other than by prohibiting logical coins altogether), because a priori any prior can believe any correlations about any coin.

Desiderata 6 (Good average performance) is discussed in Sect. 5.3.

Desiderata 7 (LIDT rewrites to UDT) depends on our notion of implementation (of the UDT agent from LIDT beliefs). If we take it to just mean "taking the current $\mathbb{Q}_n$, extrapolating it into an infinite $\mathbb{P}$, and taking actions according to it for m rounds", then indeed, under some very weak assumptions about the decision tree not directly sabotaging this, LIDT will learn to rewrite. This is, as could be expected, basically giving LIDT a chance to precommit to the policy of the next m rounds, and of course LIDT will take advantage of that, and this will temporarily patch some important and performance-decreasing dynamic instabilities in LIDT. In fact, this kind of constant refreezing of the prior seems like a natural way to juggle the benefits of finite-time updatelessness and in-the-limit learning, so the "intuitively best" versions of UDT might end up looking like this (see Sect. 5.2).

Desiderata 8 (Coherent beliefs) is satisfied by definition of our construction, and we could through similar methods instill further coherence properties.

5.2 Learning

We'll claim that the learning problems of hard-core updatelessness are actually just problems with naive Expected Value Maximization ("naive" in the sense of close-minded [5], or not introducing some deontological safety stop-gaps).

UDT can be understood as performing Expected Value Maximization over "even more stuff" than we're used to. Empirical UDT optimizes the influence channel of its algorithm, and tries to optimize the fact of which empirical world it inhabits. Logical UDT goes even further, optimizing only over different possible logical worlds. These agents can be understood as performing very risky bets. Even in the Empirical case, as a fundamental feature of UDT, our agent can be forever stuck in a horrible situation by its own design, just because it assigned low enough probability to this world in its prior (which might have been well-justified, to the extent priors can be justified).

For example, say **Ω** will offer us infinite Empirical counterfactual muggings on one and the same empirical coin. Including time-penalized utility so as for infinities not to swamp our computations, UDT might well take this mugging if the trade-offs work out. If the coin toss turns up the wrong way, UDT will keep losing utility for eternity, of its own choice. That is, in building UDT, it's as if we had given the prior a chance to precommit to an infinite policy, and UDT keeps satisfying that commitment forever.

And indeed, from within EVM, who could blame UDT?[25] But from some different intuitions, maybe we want to avoid this unnecessary infinite lock-out, even if there's a bigger infinity awaiting us in another world that never was.

The above example seems egregiously risky, but in reality, in most normal situations (a randomly sampled "fair" decision tree) there will be many small pieces of information that our agent didn't think enough to obtain, and that, when locking in its prior preventing it from ever attaining that information, amount to a lost counterfactual mugging. That's why the version of UDT that will look better to humans will probably combine finite-time strategic updatelessness with hard-coded eventual learning (although it will certainly update in more fine-grained ways than the LIDT above).

We can ensure Eventual Updating (= Learning) by hard-coding that into any prior. But this will sometimes trash around arbitrarily much knowledge from it. Indeed, consider a prior $\mathbb{P}$ which believes $\mathbb{P}(\varphi) = p$. If it also has advanced enough understanding of how decision trees work and basic math, it will know that, whenever it's offered a Logical counterfactual mugging on φ with a high enough trade-off relative to p, it should take it. That's how the Expected Value calculation will turn out, every time, if $\mathbb{P}$ also believes that this behavior doesn't have any indirect catastrophic consequences elsewhere (and this will usually be the case).
Its belief that the Expected Value is higher (from its subjective perspective) by Paying up is correct. And nonetheless, we will somehow need to trash it if we want eventual updating. We can do this by aggressively hard-coding the belief that something catastrophic will happen if it doesn't update (which is false). Or by more naturally making it believe that, after some round, its counterfactual actions are no longer being observed, no funny business is going on, and it can be updateful without any worries (which is false if $\mathbf{\Omega}$ truly offers this mugging infinitely often). But some knowledge will have to be trashed.

We continue looking for cognitive moves that might be natural and privileged here, but for now nothing has seemed more natural and satisfactory than trashing some of that knowledge as a compromise between updatelessness and learning. And this can mostly be understood as an enhanced and more interpretable version of Policy Selection [6]. In fact, on can understand the spectrum from non-updating UDT to eventually-updating Policy Selection as a fundamental representation of strategic updatelessness vs learning, or instrumentality vs epistemics.[26]

5.3 Performance

As hinted at above, it seems like we can't ask for worst-case performance guarantees, because

1. Adversarial decision trees exist for every $\mathbf{A}$.
2. We'd thus like to introduce a notion of <u>fair</u> tree so that this doesn't happen, but we don't know how to do that due to the existence of "evil" logical coins (that simulate the agent) that can steer the agent's behavior (see the discussion of Desiderata 4 in Sect. 5.1).

[25] A further avenue we plan to explore is how, for example, Kelly Betting changes this picture. But we expect radically new ideas to be necessary to integrate these two apparently conflicting intuitions.

[26] Another small piece of evidence that we might need to "add something on top" of the probability distribution comes from the realization that probability distributions cannot distinguish between the Happy Dance problem [7] from Newcomb's. That is, exactly what makes the agent act correctly in Newcomb, makes it act incorrectly in Happy Dance. For now, we just rely on "advanced enough Inductor states" (or especially, the repeated refreezing of the prior) to eventually notice, out of trial and error alone, the optimal strategy for Happy Dance.

We thus turn to average (across decision trees) performance, in which the many different logical coins are quantified over, so that hopefully the "evil" coins constitute a small enough share.[27]

Even in this case, some baseline and more structural (as in, relating to the structure of the decision tree instead of the content of the logical coins) fairness conditions might be of help. The first one is a straight-forward analogue to the learning traps of [8].

Definition 3. A decision tree $\bar{T}$ satisfies the *catastrophe-free assumption* if

1. For any node i, there is an infinite branch above i whose accumulated utility per node dominates that of any other branch in the whole tree.
2. For each $\bar{o}$ and a in the tree, there are only finitely many entanglements of the form $\mathbb{P}_i((\neg)\mathbf{A}(\bar{o}) = a \mid \varphi)$.

That is, the structure always permits for **A** to learn the right behavior and obtain the highest possible reward in the limit (since there's always a path for that, and $\mathbf{\Omega}$ doesn't infinitely punish a single action).

As mentioned above, this is not a truly complete catastrophe-free assumption since, even if it's not reflected in the syntactic structure of the tree, an infinite sequence of logical coins could indeed infinitely punish a single action (at least from the subjective perspective of the agent), and we don't know how to assume that away.

The next fairness assumption reflects a short-coming of our construction above. Since our construction above maximizes Expected Value with respect to single action-points (instead of global policies), it's still missing some forms of coordination. This is just a bug of the original formulation of UDT, called UDT1.0, and can be fixed by maximizing over policies, called UDT1.1 [9]. There are straight-forward ways to extend our construction to attain this (see Appendix B), but the following assumption can nonetheless be useful.

Definition 4. A decision tree $\bar{T}$ satisfies the *coordination-free assumption* if, whenever i and j are incomparable **A**-decision nodes (and $\bar{o}_i$, $\bar{o}_j$ observation histories ending at those nodes), and $a_i^1, a_i^2, a_j^1, a_j^2$ different actions available at i and j respectively with $\text{Future}(a_i^1) \neq \text{Future}(a_i^2)$, then none of the two include an entanglement of the form $\mathbb{P}_n((\neg)\mathbf{A}(\bar{o}_j) = a_j^l | \varphi)$.

That is, in no situation does an instance of our agent have to "guess" what another one might have done, and instead the best actions are independent of this. Again, these undesirable situations could still come about through "evil" logical coins, and we don't know how to assume those away.

Now we present a way of taking the averages. Say $\bar{U}_n(\mathbf{A})$ is the average reward **A** obtains across all decision trees of depth n, with at most n total positive reward per node, and logical coins of length at most n.

Definition 5 (Comparing averages)**.** **A** does *better on average* than $\mathbf{A}'$ if $\bar{U}_n(\mathbf{A})$ eventually dominates $\bar{U}_n(\mathbf{A}')$ (as functions of n). Alternatively, we could impose stricter bounds on reward, and compute a finite $\lim_n \bar{U}_n(\mathbf{A})$ which we compare.

We have not yet proven results of this shape, but there's a strong intuition that eventual learning will be privileged in this setting, since across "a random enough set of trees" it would seem better to learn and

[27] Better said, since the "evilness" of a coin actually resides in **A**'s belief of its correlation to its behavior, we hope that with such a varied pool of logical coins some **A**'s are conservative enough about correlations that this doesn't happen often.

adapt to the situation than to abide by some high-risk bets (that have a boundedly high reward anyway) that only rarely pay out. This starts looking more like an impossibility result for non-learning (see next section).

While such a result would be intuitively meaningful, we do worry that we have just "baked in" the right prior over worlds (decision trees). Our computation of the average implicitly assumes all decision trees have the same importance (a uniform prior). But this uniform prior doesn't exist as a probability distribution (all priors are eventually simplicity biased), so any agent with a probability distribution over decision trees will eventually "disagree arbitrarily much" with our way of weighing trees. So it's not surprising that it scores poorly.

5.4 Impossibility results

We would like our impossibility results to be "internals-agnostic". That is, for them to speak solely of external observed behavior. For example, we'd like to do something as follows.

Definition 6. We say **A** *doesn't learn* if, for any decision tree, when **A** faces (at two different times) two identical situations in different parts of that one tree, it takes the same actions.

Conjecture 1 (Sought-for impossibility result)**.** *Suppose* **A** *doesn't learn. Then, another* $\mathbf{A}'$ *that does learn does better on average.*

The problem is again that we don't know how to define identical due to problems with logical uncertainty. Indeed, say an **A** faces two structurally identical finite sets of decision problems in two different parts of a same decision tree. And say it behaves differently on them. How do we know that happened because it has "learned", and not because there's some evil coin in some other part of the tree that would have punished (from the agent's perspective) behaving "locally optimally" in both situations?

So it seems like, at least for now, we'll have to look at internals. We turn to that now.

For any LI $\mathbb{Q}$, and n, consider these two algorithms. **A** (when facing a decision tree of depth n) just runs $\mathbb{Q}$ for n steps, and then freezes that prior and uses it forever. $\mathbf{A}_f$ does the same, but hard-codes into the prior "after $f(n)$ steps, become updateful". Notice **A** and $\mathbf{A}_{n+1}$ always take the same actions (but they might not perform exactly equivalently, since their code is different). Instead of $\mathbf{A}_f$, we equivalently could be using $\mathbf{A}_{\mathbb{P}}$ with a $\mathbb{P}$ that ensures eventual updating (or fancier constructions with periodical re-freezing of the prior), but that's more nuanced to define and unnecessary for the purpose of these results.

Conjecture 2 (A humbler sought-for impossibility result)**.** *There is an* f *such that* $\mathbf{A}_f$ *does better than* **A** *on average.*

We haven't proven this yet either (and it might turn out to be quite hard to prove), but we suspect it's true for many f (for example $n/2$) by the reasoning above.

6 Conclusion

We have set up a framework to dynamically approximate logically uncertain environments. This collapses logical uncertainty to predicting and influencing the behavior of Logical Inductors, and presents desirable learning-theoretic properties. Nonetheless, the philosophical core of logical uncertainty (and especially the non-dynamic nature of one-shot situations) remain problematic. Even pragmatically

speaking, the mind-dependence of logical correlations doesn't allow us to define some natural notions.

In this framework we have implemented the basic proposals for a UDT agent. Technical advancements in modifying Logical Inductors have been required to ensure the kind of detailed reasoning about the future we needed our agent to perform. The inductive extrapolations of beliefs are also a method interesting on its own sake.

While the agents present the expected up-sides of strategic updatelessness, we haven't found any unexpected routes around the Learning vs Updatelessness trade-off. On the contrary, it seems like the most desirable versions will need to embrace this trade-off, and consciously integrate the finite-time benefits of strategic updatelessness with the in-the-limit benefits of learning.

Going forward, we'll continue working on this space of agents, trying to find natural integrations of instrumentality and epistemics. There are plenty of avenues, both central to this enterprise and radically different ideas, that we have left to pursue.

We would also like to obtain a better understanding of this notion of "desirability" that we use to favor learning (and cannot be explained from within Expected Value Maximization). Here the bottleneck seems a better understanding of logical uncertainty (especially, a more realist picture of possible logical worlds). We have a hunch that something worth discovering exists in this direction.

Acknowledgments

I am very grateful to Abram for his excellent mentorship, great mood, and his altruistic sharing of knowledge. Thank you also to Caspar Oesterheld, Sylvester Kollin, Jesse Clifton, Sahil Kulshrestha, Cecilia Wood, Daniel Herrmann, Clem von Stengel, Nicolas Macé and Brady Pelkey for valuable conversations on the topic of my work. And thank you of course to the PIBBSS team and fellows for an unforgettable summer.

References

[1] Scott Garrabrant (2017) *Two Major Obstacles for Logical Inductor Decision Theory*, AI Alignment Forum.

[2] Scott Garrabrant et al. (2016), *Logical Induction*, arXiv pre-print.

[3] Alex Appel (2018), *What are Universal Inductors, Again?*, AI Alignment Forum.

[4] Will Sawin, Abram Demski (2013), *Computable probability distributions which converge on believing true Π_1 sentences will disbelieve true Π_2 sentences* , Technical note.

[5] Jesse Clifton et al. (2023), *Open-minded updatelessness*, AI Alignment Forum.

[6] Abram Demski (2017), *Policy selection solves most problems*, AI Alignment Forum.

[7] Abram Demski (2017), *Policy selection solves most problems*, AI Alignment Forum.

[8] Caspar Oesterheld et al. (2023), *A theory of bounded inductive rationality* , Theoretical Aspects of Rationality & Knowledge.

[9] Abram Demski (2019), *Conceptual Problems with UDT and Policy Selection*, AI Alignment Forum.

[10] Mark Xu (2020), *The Solomonoff Prior is Malign*, AI Alignment Forum.

A Action-based Logical Inductors, and Embeddedness

There is a way to modify our setting so that $\mathbf{\Omega}$ only takes into account our chosen actions, not other details of the setup like our code. This might a priori seem useful as a fairness assumption.

We can achieve this by introducing *Action-based Logical Inductors*. We just introduce a syntactic variable $\mathbf{A}_m$ representing the action taken in round m, and the traders' observations will be just those actions. And then, $\mathbf{\Omega}$ doesn't check sentences including a definition of $\mathbf{A}$'s algorithm (since it doesn't know what it is), and instead only consults predictions about these syntactic variables.

Intuitively, this will amount to the Logical Inductor not only dealing with logical uncertainty, but also coming up with different hypotheses as to what algorithm ($\mathbf{A}$) is producing those actions. For example, as much math as it might know about logic[28], if it has seen only 5 actions output by $\mathbf{A}$, there are still a myriad different algorithms that could have produced this. This is equivalent to Solomonoff induction on the Turing Machine that's producing its observations (the actions).

The problem (and why this is not a good fairness assumption) is that, in this framework, we've cut off and failed to represent many interesting aspects of logical uncertainty.

Indeed, now we're no longer dealing with logical counterfactuals: we are dealing with perfectly defined observation counterfactuals (just feed the Logical Inductor the different set of actions). And this can be exploited by $\mathbf{A}$: Since the only thing that can affect the LI are actions, (a computationally strong enough) $\mathbf{A}$ can just straight-forwardly argmax over sequences of actions.
This doesn't mean that the actions will not look as if implementing updateless considerations. Maybe the sequence of actions that maximizes reward (in a long enough decision tree) is to always pay up in counterfactual mugging, so that $\mathbf{\Omega}$'s Action-based LI is convinced that we are likely to do that, and we obtain the high payoff in future muggings. But in reality, all that's going on is that our $\mathbf{A}$ is updating completely (and doing a brute-force search), and implementing the action policy that best exploits the tree. It is only using the LI as one more lever in the environment which the actions can affect.[29]

What's going on here is that we've moved the "Embeddedness" boundary.
Throughout this work, we (as researchers) where playing a non-embedded role: We are thinking about decision trees, and about $\mathbf{A}$'s, and about how they interact. And we know that our thoughts are not embedded in the sense that the only way the affect the decision tree is through the choice of $\mathbf{A}$.[30] That's why, from our perspective, we could just "argmax $\mathbf{A}$" (choose the algorithm that performs best in this decision tree, or across decision trees. And indeed, our research here can be understood as an approxi-

[28]In fact, we could also throw away Logical Induction completely and use logical omniscience, stipulating mathematically that $\mathbf{\Omega}$'s opinions on $\mathbf{A}$ are just the (uncomputable) weighted sum (by some simplicity prior over programs) of all programs that could have produced the actions thus far observed.

[29]In actuality, if we built $\mathbf{A}$ as above, it would still "subjectively believe" that it can influence the outcome of some logical coins or the LI evolution (not only through actions), and thus would not just straightforwardly argmax action policies. But this construction stops making sense, because we have a strong notion of optimality: the $\mathbf{A}$ that argmaxes always obtains the highest reward possible (when we update on the actual outcome of logical coins).

[30]In fact, this is a meaningful analogy to the realistic situation, in which we assume we (as designers, designing an agent) are free to think about the environment and the consequences of designing this or that agent. That is, an adversary's decisions depend only on $\mathbf{A}$, not our thoughts or other unrelated actions. It can be argued that otherwise the situation becomes intractable, and these are some sane minimal assumptions.

mation (using our conceptual tools) to "argmaxing **A**" (finding the best UDT algorithm), since we don't have access to unbounded compute to perform the argmax as a brute-force search.
What happens when we make our LI action-based is that we move this Embededdness boundary: Now the algorithm **A** is playing our role (the researchers, the designers), and it can freely "argmax $\bar{a}$ (the action sequence)".

We interpreted this realization as a push towards "more real embeddedness" (that is, really allowing $\mathbf{\Omega}$ to look at **A**'s code). But in that setting we face some real problems of logical uncertainty, like not being able to draw a boundary around "evil" logical coins.

Instead, we could have interpreted it as a push towards starting to look at Computational Complexity: Yes, **A** can just computably argmax over policies, and that's boring. But what if it didn't have the resources to perform that brute search? We'll look into this in the future.

B Implementing UDT1.1

The below is a way to obtain in-the-limit UDT1.1 behavior.

Basic idea: For further and further nodes in the decision tree, we condition $\mathbb{P}$ on larger and larger policies (instead of single actions). For example, for the first 10 rounds we just condition on single actions as usual, but then, from round 10 to round 20, we condition on all possible conjunctions of actions taken in those rounds (across all possible worlds). And then from 20 to 120, etc. Given some assumptions about

1. not getting infinitely screwed by early actions (catastrophe-free),
2. and not being "so unlucky" that time and time again the important entanglements land between the different unconnected policy regions, despite these boundaries having density 0,

this increasing time horizon will not be a problem.

We still have to solve (by using these increasing policy horizons) the coordination problem. Imagine we face the canonical example: our agent will see 1 or 2, and output A or B. If the two counterfactuals output different letters, they win. Imagine further that both agents in both counterfactuals (or equivalently, the agent prior to counterfactual splitting) completely understand the structure of the problem, so that they know A-B and B-A are exactly as good, and A-A and B-B exactly as bad. How do they coordinate? Well, if as in our framework both agents (or equivalently, the precursor agent) have complete access to the same representation of the decision tree, they can just choose the lexicographically first policy (as UDT1.1 does).

The problem appears when, as happens in the real world, the agents only receive observations, and have to build their representations of decision trees. A same situation can be represented by different decision trees, for example, one where the 1 counterfactual is left, and another where it is right. If we could ensure the agents share the same representation of this isomorphic decision trees (that is, they pick the same representative from the reference class), they could just choose the lexicographically first policy. But this (choosing the same representative tree) is easy to obtain: say that the precursor agent (or equivalently, the UDT prior) has settled on a method to choose a representative. For example, they use a lexicographical order on *observations* (like the bits on a picture), and use that to decide between decision

trees (something like "the lexicographically first observations go as to the left of the tree as possible").

Can we nonetheless do better? $\mathbb{P}$ is an infinite distribution on all logical sentences (despite its information seed being the finite $\mathbb{Q}_k$). Instead of increasing the finite policy horizon as above, what if we just take the infinite policy horizon from the start? We can indeed "condition" $\mathbb{P}$ on an infinite conjunction (of action points), by taking the limit of the conditional probability (about, say, total U obtained) for larger finite initial segments of it. A priori this might not converge... but given our assumption that each branch has a finite amount of utility, PLUS these amounts are bounded, I think it does always converge! (Since arbitrarily far-off nodes must yield an arbitrarily small change in U.) Also, taking an arbitrary limit is not computable, but given the "information that has gone into the prior" is finite, probably some decidable proof system will prove the value of the limit (by just taking into account all of the finite information).
We might worry that this is just "letting the earliest prior decide all actions, without updating in any way, and so this doesn't learn". Again, I think we can edit the prior to hard-code something like "I don't yet know what the best action here will be, but (maybe because I believe strongly that there are no dangerous correlations) it will be at least as updateful as looking at $\mathbb{Q}_n$, and implementing this same procedure of infinite policy selection for the infinite tail of that part of the tree".

C Utility

What do we mean by the U when we argmax $\mathbb{E}(U|\mathbf{A}(\bar{o}) = a)$?

In a finite tree it can be just total utility, but in an infinite tree we've been assuming this is just time-penalized utility, penalization starting at round 1 (with penalization sharp enough that the sum is always finite[31]). This might not feel too satisfactory, since our agent will truly trade a penny now for bankruptcy later. In the RL setting this penalization changes dynamically (always starting at the present round), but that's literally changing U every round, and would reintroduce dynamic instability.

We also note that, instead of taking any decision tree and implementing an arbitrarily sharp time penalization, it's probably more natural to just assume that in our decision tree every infinite branch sums to just a finite amount of utility (and also, if we allow for infinitely many successor nodes, that the utilities of any successor set are bounded).

We can also consider maximizing the mathematical limit of average reward per round. While this still has problems with infinity (maybe more than time-penalization), it feels more natural for our non-dynamic setting. Our agent would be correctly thinking about infinity, and caring about what to do to position itself in the best situation for eternity. This should be the limit of taking $\delta \to 0$.

There might be other preference orderings on outcomes that don't route through a scalar utility.

[31] We need not only this, but also that the agent knows it, to ensure Expected Value computations are not swamped by infinities. We can just assume this is the case for $\mathbb{P}$, or hard-code it like learning (by trashing some opinions of the Logical Inductor, but this time we will actually be improving their accuracy).

D The heart of logical uncertainty

With updatelessness we want to be fair to other worlds. But other logical worlds don't seem "as real" as other empirical worlds. Empirical worlds can be understood "on their own terms", thanks to logical omniscience. They are "a real possibility". On the contrary, logical worlds cannot be understood "on their own terms", because we are using "real-world logic" (how computations actually turn out in our calculations) to understand and define them, and so this remnant of "non-agnostic / non-uncertain ground" remains.
For example, in LIs, although we are uncertain about derivability from PA, the traders we implement are still "computed using real-world computations" (that is, they are real Turing Machines, implementing real computations). Computations inside traders are the "certain ground" that remains. As a result, real-world computations are still favored in LIs. This happens "because of the simplicity prior that chooses Traders", but of course any prior will be a simplicity prior eventually, so it actually happens more fundamentally because of using real computations.

Indeed, the difficulty of logical uncertainty can be understood as the difficulty of drawing a boundary between the "fixed ground, or form, or shape", and the "varying content". In the empirical case, all of logic is the form, and the outcomes of empirical coins are the content. In the logical case, we have to draw form / content distinctions inside logic, and we don't know how to do that. The LI framework chooses to set the boundary at "uncertainty about derivability from PA". Our modifications of Universal Inductors to have them be coherent in different ways give "a bit more terrain to form", artificially enforcing in the system certainty about some properties of real-world logic.

I'd like to peel the layers off this ground further and further, and see what's at its heart, or alternatively when everything breaks down. The ultimate goal is for logically possible worlds to "understand themselves from within, in their own terms". But this might be philosophically impossible from our real world.

For this, I built two different preliminary models of "more-agnostic" frameworks that can account for more computational hypotheses, the first one modifying the interpretations (in the Model Theory sense) of the basic operations of a Turing Machine, and the second one more radically turning Turing Machines into arbitrary functions. They are detailed in this document.

E ω meta-levels of Policy Selection

Might there be a way to have, or approximate, ω meta-levels of deciding which information to update on? That would resonate with "the agent delegating to itself".

Consider the following Policy(). When $\mathbb{P}$ is frozen, it only considers 1 meta-level up ("which things should I update on, and how often"), as above. But at some finite time, it starts considering the 2nd meta-level ("how often, and how, should I be thinking about what to update on, and which updated knowledge should I bring to this reflection"). From there on, this 2nd meta-level is fixed, and the 1st meta-level is updated according to the 2nd meta-level's opinions. For example, maybe the 2nd meta-level is worried that some parts of the tree are very updateful and others very updateless, and so recommends re-considering how much to update (that is, re-writing the 1st level) very often. And so on until the 3rd

meta-level kicks in, etc.[32]
$\mathbb{P}$ could have the opinion that the above Policy() is good. Of course, the above Policy() doesn't pertain to any finite meta-level (it is ω-level), but it can be described by a program, or equivalently a formula. Maybe we can make the Σ object so complex, that it can describe Policies as complex as the above. That will look more and more like letting the Σ be arbitrary halting computations.

F Tricking slow Logical Inductors

Consider an infinite Decision Tree (that is, an $\mathbf{\Omega}$) such that, at every node, our agent **A** can choose between Cooperate and Defect. If Cooperate, it receives 0 utilons. If Defect, it receives $r/\mathbb{O}_n(D_n)$ utilons (where n is the current round, and $r > 0$). That is, reward is proportional to how surprised $\mathbf{\Omega}$'s logical inductor was to see a Defection. D_n is a sentence encoding "**A**'s algorithm Defects in round n".

Say **A**'s probability of Defection converges to a fraction (instead of flailing around). If **A** Defects with pseudorandom probability p/q (and does so by consulting complex enough pseudorandom coins, so that $\mathbb{O}$ can't predict them in advance), then in the limit $\mathbb{O}$ will catch on to that true fact[33], and the average reward will be $\frac{(q-p)\cdot 1+p\cdot\frac{q}{p}\cdot r}{q} = 1+r-\frac{p}{q}$. So it could seem like **A** wants to choose the smallest positive fraction possible (and so $1+r$ average reward wouldn't be attainable). But actually **A** can do fancier stuff, like choosing ever-unlikelier pseudorandom coins, thus attaining $1+r$ (which still converges).
In any event, $\mathbb{O}$ will always "notice the pattern of what **A** is doing", and so **A** won't get over $1+r$ in expectation.

Suppose instead **A** is the following algorithm: "Defect iff $r/\mathbb{O}_n(D_n) > 2n$". $\mathbb{O}$ will eventually "be aware" that $D_n \leftrightarrow r/\mathbb{O}_n(D_n) > 2n$. And nonetheless, $r/\mathbb{O}_n(D_n) > 2n$ will happen infinitely often, and so **A** will have 2 as a limit point of its average reward (which nonetheless might not converge). Indeed, suppose otherwise that, after some m, $r/\mathbb{O}_n(D_n) > 2n$ is never the case again. That is, $\mathbb{O}_n(D_n) \geq r/2n$. Then, a trader always selling 1 share of D_n each round after m exploits $\mathbb{O}$ (because the harmonic series diverges). Of course, we can pull this same trick with any number in the place of 2, thus assuring an arbitrarily high limit point of average reward.

We don't yet know if $\mathbb{O}_n(D_n)$ converges to 0, nor whether the average reward converges (we could have to wait too long for the new Defection infinitely often, so that some number < 2 is also a limit point). Of course, the average reward converging (or at least not dropping below 1 in the limit) would seem more pathological. Otherwise, it's not as straightforward to label this an improvement for **A**.

This phenomenon is interesting, but probably what we want to do for studying UDT is just "assume we can prove a priori bounds on the reward a single node provides". For example, using $\min(1/\mathbb{O}_n(D_n), 100)$ in this case.

[32] As written, when we reach the nth meta-level for the first time, the opinions on it are completely non-updated (and the below levels will update according to these opinions). We probably want to change this, by stating they should have updated a bit (even the first time they are considered), so that we get learning as we wanted.

[33] Actually, due to Logical Inductors not satisfying Gaifman Inductivity in general, this might not be true in some situations. That's a direction we still have to explore.

G Everything else

There are some interesting ideas and possible future directions not included here. Also, many of the ideas and interpretations here presented have been arrived at through a circuitous road of "this avenue works and that one doesn't", and these collateral realizations are useful technical knowledge. Just in case it is ever useful in the future, here are all my notes (that were important enough to be written in LaTeX at any point).